

AI-POWERED APPLICANT TRACKING SYSTEM

The Candidate Ranking Model

Explainable retrieval-augmented candidate matching — RAG + LLM reasoning

Replace keyword matching with semantic search. Every rank is scored, explained, and traceable to evidence.

Flutter (M3) · FastAPI · PostgreSQL + Qdrant · BGE embeddings · Provider-agnostic LLM

Project technical review — status: foundation + ingestion + search/ranking implemented

What the model is built to do

- Turn unstructured resumes into structured, searchable candidate knowledge
 - Every resume is parsed, normalized, and transformed to strict JSON before indexing
- Rank candidates against a recruiter's natural-language query or a full job description
 - Semantic matching over skills, experience, projects, certifications — not keyword grep
- Produce explainable results: bucket + score + strengths/weaknesses + reason
 - Every claim maps to a retrievable evidence chunk
- Surface hidden gems — high-potential candidates rejected by years-based screening
 - Counteract years-of-experience bias with an explicit potential-scoring model

Two actors, one core

- Applicant: upload → parse → index
- Recruiter: query → retrieve → rank

Not "chat with your PDF"

- Structured knowledge base first
- LLM is a reasoner, never a free-form source

Trust by construction

- Deterministic scores own the rank
- LLM explains; it cannot invent scores

An end-to-end retrieval-augmented ranking model

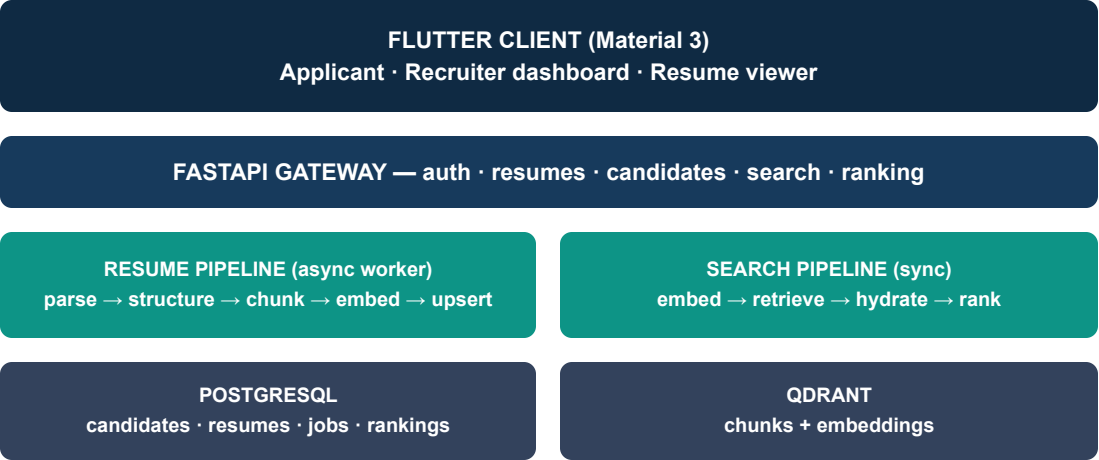
Write path — indexing

- PDF upload → validation → digital text extraction
 - OCR fallback for scanned documents (Tesseract / PaddleOCR)
- LLM structuring → strict JSON candidate profile (Pydantic-validated)
 - Skill canonicalization + years-of-experience calculator
- Semantic chunking from the structured profile, not raw text
 - Embeddings — BAAI bge-small-en-v1.5 (384-dim, CPU-friendly)
- Vector upsert into Qdrant (idempotent on chunk_id)

Read path — ranking

- Query embedding (bge query-instruction prefix at search time only)
- Vector retrieval, grouped by candidate with section diversity
 - Profile hydration from PostgreSQL (ground truth)
- Tier-1 deterministic scoring → overall score
- Tier-2 LLM reasoning over retrieved evidence only
- Bucket assignment: best / strong / hidden_gem / alternative
- Ranking persisted for audit; cached by query signature

System architecture & provider abstraction



Provider ports — swap infrastructure without changing business logic

EmbeddingProvider	OpenAI · Gemini · local BGE (ONNX) · debug
LLMProvider	OpenAI · Gemini · Ollama (Llama/Qwen/DeepSeek) · vLLM
VectorStore	Qdrant · pgvector
OCRProvider	Tesseract · PaddleOCR
ObjectStorage / FileParser	S3 · local · pdf / docx

Selected via config (settings.providers.) — adding a provider = new adapter + DI registration. Hallucination guard: the ranking LLM only sees structured profiles + retrieved chunks.*

Ranking algorithm — two-tier, hybrid, evidence-based

TIER 1 — Deterministic heuristic scoring

- skill_match — taxonomy overlap + semantic similarity
- experience_match — years ratio · domain cosine · progression
- education_match / certification_match — required criteria
- hidden_gem_score — potential signals, not years

overall = weighted sum over active dimensions

+ bounded LLM delta (clamped ± 0.08)

Buckets via transparent rules, not LLM discretion

best $\geq .80$ & cov $\geq .7$ · strong $\geq .60$ & cov $\geq .5$ · hidden_gem .45–.74 & HG $\geq .5$ · alternative

TIER 2 — LLM reasoning (evidence-constrained)

- Input: job intent + structured profile + bounded evidence pack (max N chunks)
- Output JSON: strengths · weaknesses · explanation · recommendation
- Must cite evidence_chunk_ids $\subseteq$ retrieved — else reject & regenerate
- LLM delta clamped ± 0.08 → cannot overturn a strong heuristic
- Fallback: LLM down → heuristic buckets remain, no narrative

How the model's effectiveness is measured

Retrieval & ranking quality	nDCG@20 ≥ 0.80	Ranking eval: human-labeled relevance pairs vs heuristic + LLM ranks
Pipeline reliability	Upload → INDEXED ≥ 95%	10 varied sample PDFs incl. 2 scanned must all reach INDEXED
Indexing latency	Median < 30 s	Async worker; OCR path for scanned resumes
Search latency	p95 < 2 s	Excludes cold model loads; cached by query signature
Recruiter value	Shortlist time −40%	Self-reported time-to-shortlist vs manual review
Hidden-gem value	≥ 15% shortlist → interview	Proves potential-based ranking adds value beyond years
Trust / anti-hallucination	evidence ⊆ retrieved	Every explanation claim traceable to a chunk_id
Determinism	Reproducible ranks	temperature 0 · deterministic evidence pack ordering

Golden resume set + labeled relevance pairs

Source

- Planned golden set of varied, real-world-style resumes (PDF)
- Incl. scanned documents to exercise OCR fallback path
- Human-labeled relevance pairs for ranking eval (nDCG)
- Seed directory scaffolded at backend/data/resumes (to populate)

Insights that shaped the design

- Chunking from the structured profile (not raw text) keeps chunks semantically homogeneous
- bge-small-en-v1.5: 384-dim, CPU-friendly, strong retrieval at low cost
- Query-side instruction prefix applied at search time only

Pre-processing pipeline

- File validation → PDF text extraction → OCR fallback
- Cleaning → section detection & normalization
- LLM structuring → strict JSON (schema + few-shot)
- Skill canonicalization → semantic chunking → embeddings

Dataset status

- Eval harness + golden-set assertions are spec'd in the design
- A/B harness to compare embedding models / LLM providers
- Next step: seed sample resumes and wire Qdrant to index them

How the model was designed and built

Design-first, then build

- 14-doc design package: architecture, ERD, API, AI/RAG/ranking pipelines
- Clean Architecture + repository pattern + dependency injection (SOLID)
- Ports & adapters — providers swapped via config, not code

Retrieval-augmented generation

- Indexing: parse → structure → chunk → embed → upsert (idempotent)
- Retrieval: embed query → Qdrant search → hydrate → rank
- Optional hybrid BM25 + RRF fusion for low-recall queries

Two-tier ranking by design

- Deterministic tier owns scores & buckets (transparent rules)
- LLM tier explains & refines within a bounded delta
- Deliberate decision: LLM explains "why", never decides the rank

Safety & guardrails

- Schema-validated structuring with self-healing retry
- Prompt-injection isolation; LLM never sees raw resume text
- Post-validation: evidence chunk_ids must be subset of retrieved

Vertical slices, not big-bang

- Each milestone runnable & demoable end-to-end (Phase 0 → 5)
- Definition of done: tests green, lint clean, demoable UI
- Eval-first: golden set + nDCG harness gate provider swaps

Problems encountered — and what we learned

Challenges & how they were handled

Messy / malformed resumes

- Strict JSON schema + validation, retry with error feedback, review queue

LLM hallucination

- Evidence-only prompting; chunk_id subset post-validation; raw scores stay visible

Prompt injection via resume text

- Delimiter-isolated prompts; injected strings land in text fields, never logic

Provider lock-in & cost

- Ports + adapters; local BGE default; token budgets; caching

Niche query recall

- Optional hybrid BM25 + reciprocal rank fusion

Years-of-experience bias

- Explicit hidden-gem scoring model — potential signals, code-level & auditable

Key learnings

- Deterministic rules should own the score; let the LLM explain, not decide
- Verifiability builds recruiter trust — every claim maps to a chunk
- Design-first (14 docs) avoided rework; vertical slices kept each milestone demoable
- Provider abstraction paid off: swap models/infra via config, zero business-logic changes
- Anti-bias is a product feature: hidden gems are surfaced with a clear reason

Staggered roadmap — Phase 5 and beyond

Interview question generator

- Reuse per-candidate chunks + JD to draft questions

Candidate fit score

- Persisted, long-lived match score per candidate + job

Bias detection

- Score distributions vs protected-attribute proxies; equality-of-opportunity report

Salary prediction

- Regression over structured profiles + market data

JD optimizer

- LLM suggestions over JD chunks via RAG to best-practice corpus

AI resume suggestions

- Reverse of structuring: profile → improvement suggestions

Multi-language resumes

- bge-m3 / multilingual-e5 + OCR language config

Analytics dashboard

- Funnel, pipeline-quality, and retrieval-quality metrics

Provider upgrades

- Larger embedding models behind a version gate; A/B via config

How to see the model in action

A public demo link is **not yet deployed** — the project is in early implementation. The model runs locally today:

- 1 **Start the stack**
`docker compose up` (PostgreSQL + Qdrant + worker)
- 2 **Run the API**
`uvicorn app.main:app` — health: `GET /healthz`
- 3 **Index resumes**
`POST /api/v1/resumes` → pipeline: parse → structure → embed → upsert
- 4 **Rank candidates**
`POST /api/v1/search/query` { "query": "senior Flutter engineer, payments" }
- 5 **Inspect evidence**
Response returns bucket, scores, explanation + evidence chunk_ids

Full run instructions: docs/setup-and-testing.md · Current status: Phase 0–2 scaffolded — FastAPI + Flutter + ranking engine + unit/integration tests.

SUMMARY

A semantic, explainable candidate ranking model

- ▶ Objective: resumes → structured knowledge → explainable, bias-aware ranking
- ▶ Model: RAG + two-tier hybrid (deterministic scores + evidence-constrained LLM)
- ▶ Metrics: $nDCG@20 \geq .80$ · pipeline $\geq 95\%$ · search p95 < 2 s · hidden-gem yield $\geq 15\%$
- ▶ Dataset: golden resume set + labeled relevance pairs (seeding in progress)
- ▶ Next: seed dataset, wire Qdrant indexing, LLM reasoning tier, then a live demo link